Xilu 聚合广告 SDK UniApp 接入文档 V1.0.8.6

> 仅支持 Android（App 端），H5 / 小程序端不可用。

0. 版本要求

HBuilderX 编译器版本必须与离线 SDK 版本一致，都是 **5.24**。不要更换成其他版本。

1. SDK 组成与广告类型

1.1 组成

组成	文件
桥接层	`xilu-ad-bridge/`（源码，离线打包以 project 方式依赖）
核心 SDK	`XiluSDKCore-v1.0.8.6.aar`
渠道适配器	`AdapterCSJ / GDT / BQT / KS / MS / BZ`，共 6 个
三方广告底包	穿山甲 / 优量汇 / 百青藤 / 快手 / 美数 / 倍孜 SDK
JS 工具模块	`utils/xilu-ad.js`

1.2 广告类型

类型	接入方式	页面
开屏	Module `showSplash`	`.vue`
横幅	组件 `<xilu-banner>`	`.nvue`
信息流模板	组件 `<xilu-native-express>`	`.nvue`
信息流自渲染	组件 `<xilu-native-feed>`	`.nvue`
激励视频	Module `loadRewardVideo` + `showRewardVideo`	`.vue`
插屏	Module `loadInterstitial` + `showInterstitial`	`.vue`

横幅与信息流是原生组件，只能用在 `.nvue` 页面。

2. 复制文件

第 1 步：复制 JS 与页面

```
xilu-uni-demo/utils/xilu-ad.js        →  你的项目/utils/xilu-ad.js
xilu-uni-demo/pages/ 下要用的页面      →  你的项目/pages/
```

`utils/xilu-ad.js` 必须复制，初始化和所有广告请求都走它。

页面按需复制，复制后在 `pages.json` 里登记：

demo 页面	广告类型
`pages/splash/splash.vue`	开屏
`pages/banner/banner.nvue`	横幅
`pages/native/native.nvue`	信息流模板（单条）
`pages/native-list/native-list.nvue`	信息流模板（列表，每 5 条插 1 条）
`pages/feed/feed.nvue`	信息流自渲染（单条）
`pages/feed-list/feed-list.nvue`	信息流自渲染（列表）
`pages/reward/reward.vue`	激励视频
`pages/interstitial/interstitial.vue`	插屏
`pages/index/index.vue`	首页菜单（含初始化写法）

第 2 步：复制插件包

```
xilu-uni-demo/nativeplugins/Xilu-AD/  →  你的项目/nativeplugins/Xilu-AD/
```

`android/` 下 15 个 AAR：

- 核心：`XiluSDKCore-v1.0.8.6.aar`
- 适配器：`AdapterCSJ/GDT/BQT/KS/MS/BZ-v1.0.8.6.aar`（6 个）
- 三方底包：`open_ad_sdk_7.5.1.0.aar`、`GDT_SDK.4.662.1532.aar`、`Baidu_MobAds_SDK-release_v9.450.aar`、`KS_AD_4.4.20.1.aar`、`MS_AD_2.5.7.7.aar`、`BZ_AD_5.2.1.21.aar`
- OAID：`hms-ads-identifier-3.4.62.300.aar`、`mcs-ads-identifier-1.0.2.301.aar`

离线打包不读这个目录（Android 工程用 `vendor/xilu/` + 桥接 project），它用于 HBuilderX 自定义基座 / 云打包。

第 3 步：复制离线打包工程

```
xilu-android-build/      →  你的项目/xilu-android-build/          整目录，排除 dist/、app/build/、patched-libs/
xilu-ad-bridge/          →  你的项目/xilu-ad-bridge/              整目录
vendor/xilu/             →  你的项目/vendor/xilu/                 13 个 AAR
vendor/uni-offline-sdk/  →  你的项目/vendor/uni-offline-sdk/      137 个文件（166MB）
tools/monitor/           →  你的项目/tools/monitor/               2 个文件
```

漏了会怎样

漏了	后果
`utils/xilu-ad.js`	无法初始化 SDK
`pages/`	对应广告类型测不了
`nativeplugins/Xilu-AD/package.json`	编译资源时提示插件不存在
`nativeplugins/Xilu-AD/android/` 里的 AAR	编译报找不到类 / 提示未检测到插件
`xilu-android-build/`	无法离线打包
`xilu-ad-bridge/`	编译报找不到 `:xilu-ad-bridge`
`vendor/xilu/`	编译报找不到类
`vendor/uni-offline-sdk/`	编译报找不到基座类
`tools/monitor/patch-weexcore-exit.ps1`	构建直接失败
`tools/monitor/alarmguard.c`	影响日后重编 `libalarmguard.so`

3. 复制后必须修改

3.1 插件注册（三处都要有）

1) `nativeplugins/Xilu-AD/package.json`，不用改：

```
{
    "id": "Xilu-AD",
    "version": "1.0.0",
    "name": "Xilu-AD",
    "_dp_type": "nativeplugin",
    "_dp_nativeplugin": {
        "android": {
            "minSdkVersion": "24",
            "integrateType": "aar",
            "plugins": [
                { "type": "module",    "name": "Xilu-AD",             "class": "com.xilu.sdk.uni.XiluAdModule"
                { "type": "component", "name": "xilu-banner",         "class": "com.xilu.sdk.uni.XiluBannerComp
                { "type": "component", "name": "xilu-native-express", "class": "com.xilu.sdk.uni.XiluNativeExpr
                { "type": "component", "name": "xilu-native-feed",    "class": "com.xilu.sdk.uni.XiluNativeFeed
            ]
        }
    }
}
```

2) `manifest.json`：

```
{
    "app-plus": {
        "nativePlugins": {
            "Xilu-AD": { "__plugin_version": "1.0.0" }
        },
        "distribute": {
            "android": {
                "abiFilters": ["armeabi-v7a", "arm64-v8a"]
            }
        }
    }
}
```

3) `xilu-android-build/app/src/main/assets/dcloud_uniplugins.json`，不用改，`plugins` 数组与第 1 处完全一致。

漏了第 3 处的表现：`uni.requireNativePlugin('Xilu-AD')` 返回 undefined，页面报「未检测到 Xilu-AD 插件」，组件标签无效。

3.2 配置清单

文件	改成什么
`utils/xilu-ad.js`	`APP_ID`
`utils/xilu-ad.js`	`POS` 各字段（广告位 ID）
`manifest.json`	`appid`
`manifest.json`	`nativePlugins`、`abiFilters`（见 3.1）
`xilu-android-build/app/src/main/assets/data/dcloud_control.xml`	`appid`、`appver`，必须与 `manifest.json` 一致
`xilu-android-build/app/src/main/assets/apps/__UNI__F1B6163/`	目录名改成你的 appid
`xilu-android-build/app/build.gradle`	`namespace` / `applicationId`（默认 `com.qunze.xiju`）
`xilu-android-build/keystore.properties`	复制 `keystore.properties.example` 改名，填自己的签名信息（证书要与媒体平台录入的 SHA1 一致）
`xilu-android-build/gradlew.bat`	Gradle 路径（硬编码了本机缓存路径）
`xilu-android-build/settings.gradle`	确认 `../xilu-ad-bridge` 路径正确

`dcloud_control.xml`：

```
<hbuilder>
    <apps>
        <app appid="__UNI__F1B6163" appver="1.0.0"/>
    </apps>
</hbuilder>
```

不要给 `<app>` 加 `debug` / `syncDebug`，否则启动弹「当前应用运行在自定义调试基座中」。

3.3 入口 Activity

默认支持 `io.dcloud.PandoraEntryActivity` 及其派生类，不用改动。

入口 Activity 完全不继承它时，登记一次，否则热启动开屏不生效：

```
// App 启动时调用一次，例如 Application.onCreate
XiluHotStartRouter.addHostActivity("com.your.pkg.YourEntryActivity");
```

4. 初始化

4.1 API

`init(config, callback)` —— 必须在任何广告请求之前完成。成功 `res.event === 'onSuccess'`，失败 `onFailed`。

`isInit()` —— 返回 Boolean。

4.2 参数（`INIT`）

参数	类型	必填	默认	说明
appId	String	是	-	应用 ID
debug	Boolean	否	false	置 `true` 时 `onlySupportPlatform` 才生效；上线前改回 `false`
logDebug	Boolean	否	false	打印原生日志（tag `ADXiluLog`）
agreePrivacyStrategy	Boolean	否	true	保持 `true`。置 `false` 会强制关闭 WIFI/定位/设备信息/安装列表/外部存储读取，严重影响收益
customDeviceInfo	Object	否	{}	隐私开关关闭时传入设备标识

`customDeviceInfo` 字段：`imei`、`oaid`、`vaid`、`macAddress`、`location {latitude, longitude}`。

广告请求失败提示开关：

```
export const AD_TOAST = {
    ERROR: true      // 正式包改成 false
}
```

4.3 示例

```
<script>
import { init, isInit, INIT } from '@/utils/xilu-ad'

export default {
    data() {
        return { inited: false }
    },
    onShow() {
        if (!this.inited) {
            this.inited = true
            this.initSdk()
        }
    },
    methods: {
        initSdk() {
            if (isInit()) return
            init(INIT, res => {
                if (res.event === 'onSuccess') {
                    // 可以请求广告
                } else {
                    console.error('SDK初始化失败:', res.data && res.data.error)
                }
            })
        }
    }
}
<\/script>
```

5. 各广告类型

只需要修改 `posId`，其他保持默认即可。

5.1 开屏

```
import { showSplash, POS } from '@/utils/xilu-ad'

showSplash({ posId: POS.splash }, res => {
    if (res.event === 'onADTick') {
        // 倒计时：res.data.millisUntilFinished
    } else if (res.event === 'onAdClose' || res.event === 'onAdFailed') {
        // 关闭/失败后自行返回
    }
})
```

5.2 横幅

```
<xilu-banner :posId="posId"
    @onAdRender="onRender" @onAdClose="onEvent" @onAdFailed="onEvent" />
```

属性	类型	说明
posId	String	广告位 ID（必填）

事件	说明
`onAdReceive`	拿到广告数据
`onAdRender`	渲染完成，`e.detail` 有 `width` / `height`（单位 dp）
`onAdExpose` / `onAdClick` / `onAdClose`	曝光 / 点击 / 关闭
`onAdFailed`	加载或渲染失败

容器宽高用 `onAdRender` 回传的值，不能给 0 或 1px（容器没尺寸就不发请求，表现为无广告、无回调）。

5.3 信息流模板

```
<xilu-native-express :posId="posId"
    @onAdRender="onRender" @onAdFailed="onEvent" @onRenderFailed="onEvent" />
```

属性	类型	说明
posId	String	广告位 ID（必填）

事件	说明
`onAdReceive`	拿到广告数据
`onAdRender`	渲染完成，`e.detail` 有 `width` / `height`（单位 dp）
`onAdExpose` / `onAdClick` / `onAdClose`	曝光 / 点击 / 关闭
`onAdFailed` / `onRenderFailed`	加载或渲染失败

5.4 信息流自渲染

```
<xilu-native-feed :posId="posId"
    @onAdRender="onRender" @onAdFailed="onEvent" @onRenderFailed="onEvent" />
```

属性、事件同 5.3，另加 `onVideoStart` / `onVideoPause` / `onVideoComplete` / `onVideoError`。

5.5 激励视频

```
import { loadRewardVideo, showRewardVideo, POS } from '@/utils/xilu-ad'

// 一次成功拉取的数据只能展示一次，展示后必须重新加载
loadRewardVideo({ posId: POS.reward }, res => {
    if (res.event === 'onAdReceive')      { /* 可以展示 */ }
    else if (res.event === 'onReward')    { /* 发放奖励 */ }
    else if (res.event === 'onAdClose')   { /* 需重新 load */ }
    else if (res.event === 'onAdFailed')  { /* 加载失败 */ }
})

showRewardVideo({}, res => {
    if (res.event === 'onShowFailed') { /* 无可用广告 / 未准备好 / 已失效 */ }
})
```

5.6 插屏

```
import { loadInterstitial, showInterstitial, POS } from '@/utils/xilu-ad'

loadInterstitial({ posId: POS.interstitial }, res => {
    if (res.event === 'onAdReceive')     { /* 可以展示 */ }
    else if (res.event === 'onAdClose')  { /* 需重新 load */ }
    else if (res.event === 'onAdFailed') { /* 加载失败 */ }
})

showInterstitial({}, res => {
    if (res.event === 'onShowFailed') { /* 展示失败 */ }
})
```

5.7 Module 事件

事件	说明
`onSuccess` / `onFailed`	`init` 成功 / 失败
`onAdReceive` / `onAdReady`	广告数据就绪
`onAdExpose` / `onAdClick` / `onAdClose`	曝光 / 点击 / 关闭
`onAdFailed`	加载或渲染失败
`onReward`	激励发放
`onShowFailed`	展示失败（未就绪 / 已失效 / 无填充）
`onADTick`	开屏倒计时
`onVideoCache` / `onVideoComplete` / `onVideoError`	视频缓存 / 播放完成 / 播放错误

6. 广告位 ID

```
export const POS = {
    splash: 'ek96tfdg',            // 开屏
    banner: 'uk4jsu3p',            // 横幅
    reward: 'xgpvcpp3',            // 激励视频
    interstitial: '9hmeh3af',      // 插屏
    nativeTemplate: '675bmzvg',    // 信息流模板
    nativeSelfRender: '59jmkybj'   // 信息流自渲染
}
```

全部换成你后台申请的广告位 ID。

7. 离线打包

7.1 生成 App 资源

```
# 1) 定位 cli.exe
Get-ChildItem "C:\Program Files" -Filter "HBuilderX*" -Directory

# 2) 确认项目已导入（未导入会报「项目 xxx 不存在，请先导入」）
& "<HBuilderX路径>\cli.exe" project list

# 3) 编译（命令写一行，路径含空格要加引号）
& "<HBuilderX路径>\cli.exe" publish app-android --type appResource --project "D:\project\your-project"
```

成功输出 `项目 xxx 编译成功。` + `导出成功，路径为：...unpackage\resources`。

产物在 `unpackage/resources/<appid>/www`。

7.2 覆盖资源

把 `unpackage/resources/__UNI__F1B6163/www/` 整个目录覆盖到：

```
xilu-android-build/app/src/main/assets/apps/__UNI__F1B6163/www/
```

目录名就是你的 appid。`assets/apps/` 下不要留旧 appid 的目录。

7.3 编译

```
cd xilu-android-build
gradlew.bat assembleDebug      # app/build/outputs/apk/debug/app-debug.apk
gradlew.bat assembleRelease    # app/build/outputs/apk/release/app-release.apk
```